# Noise in Quantum Computing with Qiskit

Student Lab Handout

## Session Objectives

1. Explain what *noise* is in quantum hardware and why it matters
2. Simulate noisy quantum circuits using Qiskit Aer
3. Identify gate noise, decoherence ($T_1$, $T_2$), and measurement errors
4. Compare ideal and noisy quantum circuit outputs

## Timed Agenda (75 minutes)

1. **0:00-0:05**: Introduction - why noise? (5 min)
2. **0:05-0:15**: Ideal circuit behavior (10 min)
3. **0:15-0:20**: Introduction noise models (5 min)
4. **0:20-0:35**: Gate errors (15 min)
5. **0:35-0:50**: Decoherence noise (15 min)
6. **0:50-1:05**: Measurement readout (15 min)
7. **1:05-1:15**: Real backend noise (10 min)

# 1 Introduction: Why Noise?

Quantum states are extremely fragile. Any interaction with the environment introduces errors that degrade quantum information. Current quantum computers operate in the so-called *NISQ* (Noisy Intermediate-Scale Quantum) era.

Unlike classical noise, quantum noise affects amplitudes, phases, and entanglement:

- Errors accumulate over time
- Coherence is lost
- Entanglement degrades

## Setup and Imports

We begin by importing the necessary Qiskit and Python packages.

```
import numpy as np
from qiskit import QuantumCircuit, transpile
from qiskit_aer import AerSimulator
from qiskit.visualization import plot_histogram
```

To work with noise, we import the Qiskit noise module:

```
from qiskit_aer.noise import NoiseModel
```

# 2 Ideal Reference Circuit

We first study an ideal quantum circuit, with no noise, to establish a baseline.

### Bell State Circuit

The Bell state

$$\left|\Phi^+\right\rangle = \frac{1}{\sqrt{2}}\left(|00\rangle + |11\rangle\right)$$

is created using a Hadamard gate followed by a CNOT:

```
qc = QuantumCircuit(2, 2)
qc.h(0)
qc.cx(0, 1)
qc.measure([0,1], [0,1])
qc.draw('mpl')
```

### Simulation

```
ideal_backend = AerSimulator()
transpiled_qc = transpile(qc, ideal_backend)
job = ideal_backend.run(transpiled_qc, shots=1024)
ideal_counts = job.result().get_counts()
plot_histogram(ideal_counts)
```

### Expected Result

Analytically, the expected probabilities are as follows:

| Outcome | Probability |
|---|---|
| 00 | 0.5 |
| 11 | 0.5 |
| 01 | 0 |
| 10 | 0 |

This perfect correlation will serve as a comparison for noisy simulations.

The histogram obtained from the ideal simulator represents the *theoretical, noise-free behavior* of the quantum circuit. In this setting, we assume:

- perfectly implemented gates,
- no interaction between the qubits and their environment,
- and error-free measurement.

As a consequence, the Bell circuit produces only the outcomes $|00\rangle$ and $|11\rangle$, each with probability $1/2$, reflecting perfect quantum correlations between the two qubits.

This ideal result serves as a *reference benchmark* for the rest of the lab. Any deviation observed later, such as:

- the appearance of $|01\rangle$ or $|10\rangle$ outcomes,
- unequal probabilities for $|00\rangle$ and $|11\rangle$,
- or reduced correlation between qubits,

cannot be attributed to the algorithm or circuit design, but must be caused by *noise* introduced in the simulation.

By comparing noisy results to this ideal reference, we can:

- identify which types of noise are most damaging,
- understand how errors manifest in measurement statistics,
- and build intuition for how real quantum hardware differs from theoretical models.

In other words, the ideal simulation defines the "correct answer". The rest of the lab explores how and why real-world effects cause experimental results to depart from this ideal behavior.

**Statistical fluctuations in the ideal simulation.** Although the ideal Bell state produces exact theoretical probabilities

$$P(00) = P(11) = \tfrac{1}{2},$$

the observed measurement counts obtained from the simulator (for example, 518 occurrences of $|00\rangle$ and 506 of $|11\rangle$ out of 1024 shots) are not exactly equal.

This small imbalance is *expected* and does not indicate any form of physical noise or simulation error. Even in a perfectly noiseless quantum simulator, measurement outcomes are generated by sampling a probability distribution a finite number of times. As a result, the counts fluctuate statistically from run to run.

For a two-outcome experiment with probability $p = \frac{1}{2}$ and $N = 1024$ shots, the number of outcomes follows a binomial distribution with:

$$\text{mean } \mu = Np = 512, \qquad \text{standard deviation } \sigma = \sqrt{Np(1-p)} = 16.$$

The observed deviations of a few counts from 512 are therefore well within normal statistical fluctuations. This effect is known as *shot noise* or *sampling noise*. It is fundamentally different from the physical noise studied later in this lab:

- Sampling noise arises from taking a finite number of measurements and can be reduced by increasing the number of shots.
- Physical noise (gate errors, decoherence, readout errors) originates from hardware imperfections and *cannot* be removed by increasing the number of shots.

The ideal simulation thus provides the correct reference behavior. Any systematic deviation from this pattern observed in later sections must be attributed to the introduction of physical noise models, not to statistical fluctuations.

# 3 Noise models

Noise is not the same as random "bugs" or programming mistakes. In quantum systems, noise arises from specific, identifiable physical mechanisms that act continuously during computation and measurement. Common sources include gate errors, which result from imperfect control pulses and calibration limits; decoherence, quantified by the relaxation time $T_1$ and dephasing time $T_2$, which reflects unavoidable coupling between the qubit and its environment; and measurement errors, where classical electronics or detectors misidentify the final quantum state. Unlike software bugs, these errors cannot simply be removed by correcting code; they must be modeled, characterized, and mitigated using physical insight and statistical techniques. Understanding the origin of noise is therefore essential for designing reliable quantum experiments and developing effective error-mitigation strategies.

## 3.1 Gate Noise: Depolarizing Errors

**Depolarizing noise: physical and mathematical interpretation.** Gate noise models the fact that physical quantum operations are never implemented perfectly. Control pulses have finite precision, qubits can interact weakly with their environment during operations, and unwanted couplings may occur. As a result, the actual operation applied to the qubits deviates slightly from the intended ideal gate.

A commonly used phenomenological model for such imperfections is the *depolarizing channel*, which acts on a quantum state $\rho$ as

$$\rho \longrightarrow (1-p)\,\rho + p\,\frac{I}{2^n},$$

where $p$ is the depolarizing probability, $n$ is the number of qubits, and $I$ is the identity operator on the $2^n$-dimensional Hilbert space.

This expression has a simple operational meaning:

- With probability $1-p$, the quantum gate is applied correctly and the state $\rho$ is left unchanged.
- With probability $p$, the state is replaced by the maximally mixed state $I/2^n$, which represents complete loss of information about the original quantum state.

In other words, depolarizing noise models a situation in which the qubit state is partially "randomized" during the gate operation. The maximally mixed state assigns equal probability to all basis states and contains no coherence or entanglement.

Although real hardware noise can be more structured, the depolarizing channel is widely used because:

- it is simple and easy to interpret,
- it treats all errors on an equal footing,
- it provides a convenient way to quantify overall gate quality.

Importantly, depolarizing noise does *not* correspond to a specific physical mechanism, but rather captures the net effect of many small, uncontrolled error sources acting during a gate.

**Connection to quantum circuits.** In a quantum circuit, depolarizing noise is typically applied *after* each gate. For single-qubit gates the dimension is 2 and $I/2$ represents a completely random single-qubit state. For two-qubit gates, the dimension is 4 and $I/4$ represents a completely mixed two-qubit state.

As a result:

- single-qubit gate noise slightly blurs individual qubit states,
- two-qubit gate noise rapidly destroys entanglement and correlations.

This is why noisy two-qubit gates are often the dominant source of error in NISQ-era quantum computations.

**Why noise models act on $\rho$ instead of $|\phi\rangle$.** In quantum mechanics, pure states are often represented using kets, such as $|\phi\rangle$. When a system is in a perfectly known and isolated state $|\phi\rangle$, its mathematical representation can equivalently be written as the *density matrix*

$$\rho = |\phi\rangle\langle\phi| \,.$$

However, when noise is present, as is always the case in real quantum hardware, the system can no longer be described by a single pure state. Instead, it may be in a *statistical mixture* of states, where only probabilistic information about the system is available. This more general situation requires the use of a density matrix $\rho$, which can represent both pure and mixed states.

For example:

- A pure state is represented by $\rho = |\phi\rangle \langle\phi|$.
- A mixed state, such as being in $|0\rangle$ with probability $p$ and in $|1\rangle$ with probability $1-p$, is represented by
$$\rho = p \; |0\rangle \langle 0| + (1-p) \; |1\rangle \langle 1| \,.$$

Noise processes, such as gate errors, decoherence, or depolarization, do not act on kets directly. Instead, they describe how our *knowledge* of the quantum state is degraded by uncontrolled interactions with the environment. For this reason, noise is modeled as a transformation of the density matrix:

$$\rho \;\longrightarrow\; \mathcal{E}(\rho),$$

where $\mathcal{E}$ is called a *quantum channel*. Here we use the depolarizing channel model

$$\rho \;\longrightarrow\; (1-p)\,\rho + p\,\frac{I}{2^n},$$

### Constructing a Noise Model

```python
from qiskit_aer.noise import NoiseModel, depolarizing_error

# Initialize the noise model
noise_model = NoiseModel()

# 1-qubit depolarizing error (1% error)
p1 = 0.01
error_1q = depolarizing_error(p1, 1)

# 2-qubit depolarizing error (5% error)
p2 = 0.05
error_2q = depolarizing_error(p2, 2)

# Apply errors to specific gates
noise_model.add_all_qubit_quantum_error(error_1q, ['h'])
noise_model.add_all_qubit_quantum_error(error_2q, ['cx'])
```

### Running with Noise

```python
noisy_backend = AerSimulator(noise_model=noise_model)
tqc = transpile(qc, noisy_backend)
job = noisy_backend.run(tqc, shots=1024)
noisy_counts = job.result().get_counts()

plot_histogram([ideal_counts, noisy_counts],
               legend=['Ideal', 'Noisy'])
```

### Discussion

- Why does the CNOT gate dominate the error?
- Why do incorrect outcomes appear?

**Example illustrating the action of the depolarizing channel on a Bell state.** We now explicitly apply the depolarizing channel to the density matrix of the ideal Bell state. Recall that the Bell state

$$\left|\Phi^+\right\rangle = \frac{1}{\sqrt{2}}\left(|00\rangle + |11\rangle\right)$$

has density matrix

$$\rho_{\Phi^+} = \left|\Phi^+\right\rangle\left\langle\Phi^+\right| = \frac{1}{2}\left(|00\rangle\langle 00| + |00\rangle\langle 11| + |11\rangle\langle 00| + |11\rangle\langle 11|\right). \tag{1}$$

In the computational basis $\{|00\rangle, |01\rangle, |10\rangle, |11\rangle\}$, this can be written as

$$\rho_{\Phi^+} = \frac{1}{2}\begin{pmatrix} 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 \end{pmatrix}. \tag{2}$$

For a two-qubit system ($n = 2$), the depolarizing channel acts as

$$\rho \longrightarrow (1-p)\rho + p\,\frac{I}{4}, \tag{3}$$

where $I/4$ is the maximally mixed two-qubit state,

$$\frac{I}{4} = \frac{1}{4}\big(|00\rangle\langle 00| + |01\rangle\langle 01| + |10\rangle\langle 10| + |11\rangle\langle 11|\big) = \frac{1}{4}\begin{pmatrix}1&0&0&0\\0&1&0&0\\0&0&1&0\\0&0&0&1\end{pmatrix}. \tag{4}$$

Applying Eq. (3) to $\rho_{\Phi^+}$ gives

$$\begin{aligned}\rho_{\text{depol}} &= (1-p)\rho_{\Phi^+} + p\,\frac{I}{4}\\ &= (1-p)\frac{1}{2}\begin{pmatrix}1&0&0&1\\0&0&0&0\\0&0&0&0\\1&0&0&1\end{pmatrix} + p\frac{1}{4}\begin{pmatrix}1&0&0&0\\0&1&0&0\\0&0&1&0\\0&0&0&1\end{pmatrix}.\end{aligned} \tag{5}$$

Carrying out the sum explicitly, we obtain

$$\boxed{\rho_{\text{depol}} = \begin{pmatrix}\frac{1-p}{2}+\frac{p}{4}&0&0&\frac{1-p}{2}\\0&\frac{p}{4}&0&0\\0&0&\frac{p}{4}&0\\\frac{1-p}{2}&0&0&\frac{1-p}{2}+\frac{p}{4}\end{pmatrix}}. \tag{6}$$

### Exercise: Exploring Noise Strength (10 min)

Increase the depolarizing error rate applied to the two-qubit CNOT gate and observe how the output distribution changes.

The following code runs the same Bell-state circuit for different values of the depolarizing probability $p$.

```
p2_values = [0.01, 0.05, 0.1]
results = []

for p in p2_values:
    # 1. Setup noise model
    nm = NoiseModel()
    error_2q = depolarizing_error(p, 2)
    nm.add_all_qubit_quantum_error(error_2q, ['cx'])

    # 2. Attach noise model to the backend
    noisy_backend = AerSimulator(noise_model=nm)

    # 3. Transpile for this specific backend
    transpiled_qc = transpile(qc, noisy_backend)

    # 4. Run the circuit
    job = noisy_backend.run(transpiled_qc, shots=1024)
    results.append(job.result().get_counts())
```

After running the loop, plot the histograms side by side and compare them to the ideal result.

- **Which gate dominates the error?**
  Explain why increasing noise on the CNOT gate has a much stronger effect than noise on single-qubit gates.
- **When does the result become useless?**
  At what depolarizing rate do the output probabilities no longer allow you to reliably distinguish the Bell state from random noise?

### 3.2 Decoherence: $T_1$ and $T_2$

Decoherence occurs even without applying gates. Decoherence refers to the loss of quantum information due to unavoidable interactions between a qubit and its surrounding environment. Unlike gate noise, decoherence occurs *even when no quantum gates are applied.* Simply storing a quantum state for a finite amount of time is enough for errors to appear. Two key decoherence mechanisms are characterized by the time scales $T_1$ and $T_2$.

**Energy relaxation ($T_1$).** The $T_1$ time describes *energy relaxation.* Physically, this process corresponds to a qubit in the excited state $|1\rangle$ losing energy to its environment and relaxing to the ground state $|0\rangle$:

$$|1\rangle \longrightarrow |0\rangle .$$

This can be caused by interactions with electromagnetic modes, thermal noise, or other degrees of freedom in the hardware. The probability that a qubit remains in $|1\rangle$ decays exponentially with time:

$$P(|1\rangle , t) = e^{-t/T_1}.$$

As a result, $T_1$ errors change the *population* of quantum states. In multi-qubit circuits, $T_1$ relaxation tends to increase the population of states with more zeros, such as $|00\rangle$.

**Dephasing ($T_2$).** The $T_2$ time describes *loss of phase coherence*, or dephasing. This process does not change the populations of $|0\rangle$ and $|1\rangle$, but instead destroys the relative phase between them.

For a superposition

$$\frac{1}{\sqrt{2}} \left( |0\rangle + |1\rangle \right),$$

dephasing causes the off-diagonal elements of the density matrix to decay over time:

$$\rho_{01}(t) \sim e^{-t/T_2}.$$

Physically, dephasing arises from random fluctuations in the qubit's energy splitting, often due to noise in control fields or coupling to other nearby systems. Although the probabilities of measuring $|0\rangle$ or $|1\rangle$ remain unchanged, the ability of the qubit to exhibit interference and entanglement is gradually lost.

**Key distinction between $T_1$ and $T_2$.**

- $T_1$ processes change *state populations* by driving $|1\rangle \to |0\rangle$.
- $T_2$ processes destroy *quantum coherence* and phase information without changing populations.

Both mechanisms are forms of decoherence, and both act continuously in time, even when the circuit contains no gates. This is why adding idle time (for example using delay instructions) leads to visibly worse results in the simulation.

**Why this matters for quantum computing.** Decoherence sets a fundamental limit on how long quantum information can be stored and manipulated. Even perfectly calibrated gates cannot prevent errors if the computation takes too long compared to $T_1$ and $T_2$. Understanding these processes is therefore essential for designing realistic quantum algorithms and for appreciating the need for error mitigation and quantum error correction.

### Decoherence Noise Model

```
# 1. Define strengths
gamma_T1 = 0.05   # amplitude damping strength
lambda_T2 = 0.08 # phase damping strength

# 2. Create the errors (from qiskit_aer.noise)
T1_error = amplitude_damping_error(gamma_T1)
T2_error = phase_damping_error(lambda_T2)

# 3. Build the model
decoherence_noise = NoiseModel()

# 4. Compose and apply errors (usually applied to identity or single-qubit gates)
# You can compose them using the dot product for a combined effect
combined_error = T1_error.compose(T2_error)

decoherence_noise.add_all_qubit_quantum_error(combined_error, ['id', 'rz'])
```

### Adding Delay to Expose Decoherence

```
# This qubit relaxes on a 30 micro-s timescale and dephases on a 20 micro-s timescale.
T1 = 30000      # ns
T2 = 20000      # ns
delay_time = 10000 #500  # ns

decoherence_noise = NoiseModel()

# tells the simulator what happens during that wait.
# Given a qubit with relaxation time T1 and dephasing time T2, how much does it decohere during this much time?
delay_error = thermal_relaxation_error(
    t1=T1,
    t2=T2,
    time=delay_time
)

# Add the same error model to every delay instruction on all qubits
#Whenever the circuit has a delay instruction, apply this delay_error noise model.
decoherence_noise.add_all_qubit_quantum_error(
    delay_error, ['delay']
)

# Define the circuit with a Delay
qc_decoh = QuantumCircuit(2, 2)
qc_decoh.h(0)
qc_decoh.cx(0, 1)

# Simulate waiting time using a delay of 500 nanoseconds
# 'ns' is the unit; 's', 'ms', 'us', or 'dt' are also supported
qc_decoh.delay(delay_time, unit='ns') # tells the circuit how long it waits
# "Insert a 500 ns idle period in the circuit"
# delay is clock, not a noise!

qc_decoh.measure([0, 1], [0, 1])

# Setup the Noisy Backend
# Note: Your NoiseModel must now target the 'delay' instruction
```

```
noisy_backend = AerSimulator(noise_model=decoherence_noise)

# Transpile (Optimization level 0 is still safest to keep the delay intact)
t_qc_decoh = transpile(qc_decoh, noisy_backend, optimization_level=0)
decoh_counts = noisy_backend.run(t_qc_decoh, shots=1024).result().get_counts()

# Plot
plot_histogram(
    [ideal_counts, decoh_counts],
    legend=["Ideal (No Delay)", "With Decoherence During Delay"]
)
```

### Exercise

1. **Which outcome increases due to $T_1$ relaxation?**
2. **How is this effect different from depolarizing noise?**
3. **Why does adding idle time make things worse?**

## 3.3 Measurement (Readout) Errors

Readout errors occur after computation and can distort results even if the computation was correct.

**Readout errors: why correct computations can look wrong.** Readout (or measurement) errors occur at the very final stage of a quantum computation, when the state of each qubit is measured and converted into classical bits. Importantly, these errors occur *after* all quantum gates have been applied. As a result, even a perfectly executed quantum algorithm can produce incorrect-looking results due solely to imperfect measurement.

Physically, readout errors arise because measuring a qubit is a complex process. In most quantum hardware platforms, the measurement does not directly reveal the state $|0\rangle$ or $|1\rangle$, but instead relies on an indirect signal (such as emitted photons, resonator frequencies, or currents) that must be interpreted by classical electronics. Noise, finite resolution, and detector imperfections can cause this interpretation to be wrong.

**Classical misclassification of quantum states.** A readout error can be summarized as a misclassification event:

- the qubit is physically in state $|0\rangle$, but is read as $|1\rangle$,
- or the qubit is physically in state $|1\rangle$, but is read as $|0\rangle$.

These errors do not modify the quantum state before measurement; they only affect how the measurement outcome is recorded.

**Readout errors as a probability matrix.** Readout noise is typically modeled using a classical confusion (or assignment) matrix,

$$M = \begin{pmatrix} P(0 \mid 0) & P(1 \mid 0) \\ P(0 \mid 1) & P(1 \mid 1) \end{pmatrix}, \tag{7}$$

where $P(i \mid j)$ denotes the probability of measuring outcome $i$ given that the true quantum state was $|j\rangle$.

In an ideal measurement setup,

$$P(0 \mid 0) = P(1 \mid 1) = 1, \qquad P(1 \mid 0) = P(0 \mid 1) = 0,$$

but on real hardware these conditions are never exactly satisfied.

However, unlike gate and decoherence errors, readout errors can often be mitigated using classical post-processing techniques that estimate and invert the confusion matrix.

```
from qiskit_aer.noise import NoiseModel, ReadoutError

# Define the assignment matrix
# [[P(0|0), P(1|0)],
#  [P(0|1), P(1|1)]]
probabilities = [[0.9, 0.1],
                 [0.2, 0.8]]

readout_error = ReadoutError(probabilities)

# Initialize and add to model
noise_model_ro = NoiseModel()
noise_model_ro.add_all_qubit_readout_error(readout_error)

# Simple 1-qubit circuit: prepare |0>
qc = QuantumCircuit(1, 1)
qc.measure(0, 0)   # noisy measurement under noise_model_ro

# Run with noisy simulator
backend = AerSimulator(noise_model=noise_model_ro)
tqc = transpile(qc, backend)
result = backend.run(tqc, shots=1024).result()
counts = result.get_counts()

# Plot histogram
plot_histogram(counts, title="1-qubit measurement with readout error")
```

## 4 Using a Real Backend Noise Profile (Optional but Powerful)

So far, we have constructed artificial noise models to study specific error mechanisms in isolation. While this approach is pedagogically useful, real quantum hardware exhibits a complex combination of gate errors, decoherence, and readout imperfections that vary from qubit to qubit and evolve over time.

Qiskit allows us to incorporate *realistic noise profiles* derived from actual quantum devices. This enables us to simulate how a given circuit would behave on real hardware, without needing to submit a job to a physical device or wait in a queue.

**Why use a real backend noise model?** A real backend noise profile captures:

- gate-dependent error rates,
- qubit-specific decoherence times ($T_1$, $T_2$),
- asymmetric readout errors,
- realistic device connectivity and scheduling effects.

As a result, the simulated output closely resembles the real experimental data and highlights the challenges of running algorithms on noisy quantum hardware.

### Simulating a Real Device with a Fake Backend

Qiskit provides *fake backends* that mimic the noise characteristics of real IBM Quantum devices at a fixed calibration snapshot.

```
from qiskit_ibm_runtime.fake_provider import FakeVigoV2
from qiskit_aer import AerSimulator
from qiskit import transpile
from qiskit.visualization import plot_histogram

# 1. Load a realistic fake backend
fake_backend = FakeVigoV2()

# 2. Create a noisy simulator from the backend
realistic_simulator = AerSimulator.from_backend(fake_backend)

# 3. Transpile the circuit for this backend
transpiled_qc = transpile(qc, realistic_simulator)

# 4. Run the simulation
job = realistic_simulator.run(transpiled_qc, shots=1024)
realistic_counts = job.result().get_counts()

# 5. Plot the results
plot_histogram(realistic_counts)
```

The resulting histogram represents how the Bell-state circuit would likely behave on physical hardware with realistic noise and calibration constraints.

**Interpretation.** Compared to the ideal simulation, you should observe:

- degraded Bell-state correlations,
- asymmetric outcome probabilities,
- contributions from multiple noise sources acting together.

These effects are not due to a single error mechanism, but to the combined impact of gate imperfections, decoherence, and measurement errors.

### Summary of Noise Types in This Lab

| Noise Type | Physical Meaning | Modeled How |
|---|---|---|
| Gate Noise | Imperfect quantum operations | Depolarizing channels |
| Decoherence | Environmental coupling | Amplitude & phase damping |
| Measurement Noise | Readout misclassification | `ReadoutError` |

Using a realistic backend noise profile demonstrates why running quantum algorithms on present-day hardware is fundamentally different from executing them on ideal simulators. It also motivates the development of error mitigation techniques, which aim to extract useful information from noisy quantum computations without requiring full error correction.

## 5 Reflection Questions

- Which noise source was most damaging?
- Why are two-qubit gates problematic?
- Why does increasing the number of shots not fix noise?
- Why is error mitigation necessary?

## Key Takeaway

Noise is not a software bug; it is a fundamental physical limitation. Understanding noise is the first step toward building reliable quantum algorithms.